MATP: A Unified Memory-Augmented Tool Policy Framework with Causal State Invalidation, Temporal Decay, and Macro Distillation for Long-Horizon Autonomous Agents

Rahul Saravanan(231IT054), Shishir Hegde(231IT067), and Ashab Muhammed(231IT011)

Abstract—Autonomous tool-using agents powered by large language models (LLMs) suffer from three fundamental failure modes in extended operational workflows: (1) catastrophic forgetting of early architectural invariants under FIFO context eviction; (2) retrieval of obsolete or stale mutable state vectors under unversioned semantic search (naive RAG); and (3) procedural thrashing from unguided tool selection and recurrent parameter schema violations. We present MATP (Memory-Augmented Tool Policy), a unified, single-repository system that resolves these pathologies through a synthesized multi-tier memory architecture exposed over the Model Context Protocol (MCP). MATP integrates three complementary foundations into a single durable SQLite-backed substrate: (i) a version-aware temporal memory buffer implementing a piecewise retrieval function $R(q, v_i)$ that suppresses superseded versions and attenuates historical state via causal logical-clock decay with online adaptive feedback calibration; (ii) an atomic trajectory distillation engine and cross-session causal Directed Acyclic Graph (DAG) that propagates invalidation across mutable resources (db, file, api, git) and mines hierarchical composite macro-actions; and (iii) a dynamic Bayesian tool recommendation matrix with Laplace-smoothed Beta posterior confidence and self-healing parameter auto-correction hints. Evaluated on long-horizon schema refactoring scenarios, MATP completely eliminates stale-state leakage (0% vs. 40% for naive RAG) while achieving 100% active-state retrieval precision. Trajectory distillation achieves a 92.6% token reduction (13.5× compression) while retaining 88.0% of essential fact triples (vs. 37.3% under raw truncation). The entire monorepo operates with zero GPU dependencies, executing end-to-end vector retrieval, causal invalidation, and policy recommendations in under 5 ms.

Index Terms—Autonomous Agents, Semantic Memory, Causal DAG, Invalidation Propagation, Model Context Protocol, Macro Distillation, Tool Policy Learning, Temporal Decay.

1 Introduction

Autonomous agents powered by Large Language Models (LLMs) increasingly perform long-horizon software engineering, administrative automation, and multi-step computational reasoning tasks. In these settings, an agent interacts with external environments through a continuous cycle of intent formulation, tool execution (e.g., file system modifications, SQL queries, REST API requests, shell commands), and environment observation. Despite rapid advances in underlying transformer reasoning capabilities, LLM-based agentic workflows exhibit three systemic points of failure as task horizons scale beyond ten to twenty discrete turns:

1) Catastrophic Context Eviction (The FIFO Trap): Due to finite token context windows and quadratic attention costs, agent harnesses typically apply sliding-window or First-In-First-Out (FIFO) context pruning. When early critical state—such as foundational database DDL schemas, cryptographic secrets, or baseline architectural invariants—is evicted from working memory, the agent loses track of fundamental constraints established in early phases of the task.
2) Stale-State Hallucination (The Naive RAG Trap): Standard Retrieval-Augmented Generation (RAG) indexes environment observations into flat vector spaces using semantic embeddings. However, naive cosine similarity contains no representation of temporal succession or state mutability. When an agent alters a file or mutates a relational table, an unversioned vector database continues to retrieve high-cosine embeddings of the earlier, superseded state, causing the agent to execute code against obsolete schemas.
3) Procedural Thrashing and Parameter Schema Failures: Even when supplied with static system instructions, agents repeatedly attempt invalid tool combinations or submit malformed JSON payloads. Static prompts lack dynamic feedback on which tools historically succeed under specific sub-goals, leading to repeated trial-and-error loops that exhaust context limits.

To overcome these structural limitations, three research initiatives were initially undertaken at NITK Surathkal: (1) a temporal and state-aware semantic memory buffer with causal logical clocks and adaptive decay; (2) a background trajectory distillation engine with cross-session causal DAG

• R. Saravanan, S. Hegde, and A. Muhammed are with the Department of Information Technology, National Institute of Technology Karnataka (NITK), Surathkal, Mangalore 575025, India.
E-mails: rahul.skyscc@gmail.com, shishirsh2005@gmail.com, ashabcode@gmail.com.

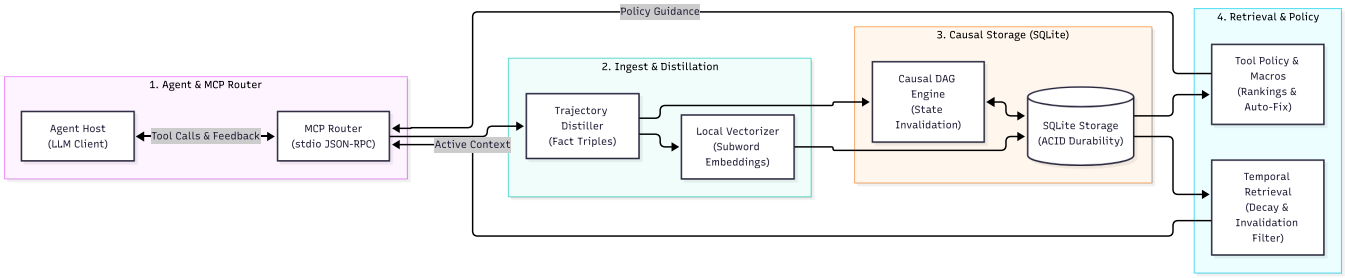


Fig. 1: Overall Architectural Blueprint of the Unified MATP Monorepo. Host LLM agents interact through a unified stdio Model Context Protocol (MCP) router. Ingested execution trajectories undergo background distillation and resource canonicalization before persistence into SQLite. The causal DAG propagates version-aware invalidations, the temporal decay engine modulates ranking via logical clocks, and the policy engine provides prompt-injectable tool matrices and auto-correction hints.

invalidation and hierarchical macro-action mining; and (3) a dynamic Model Context Protocol (MCP) tool policy engine with Bayesian confidence scoring and self-healing parameter error correction.

In this work, we present the consolidated MATP Monorepo, a fully unified architectural and software deliverable that combines all three distinct research contributions into a single, cohesive, zero-external-dependency framework. We show that these three techniques are deeply synergistic: trajectory distillation extracts canonical resource interactions that feed the causal DAG; the causal DAG establishes hard validity boundaries that filter the vector store; logical clocks apply temporal decay to active nodes; and the resulting clean historical records populate the dynamic tool recommendation matrix and self-healing error database.

2 System Architecture and Storage Design

The unified MATP monorepo is engineered as a modular, local-first system adhering to the open Model Context Protocol (MCP) standard. Figure 1 illustrates the system architecture and dataflow across all operational layers.

2.1 Protocol Boundary and Agent Orchestration

The agent host environment communicates with MATP through a standard JSON-RPC 2.0 stdio interface. This decouples the agent runtime (e.g., Claude Desktop, Cursor, Roo Code, custom LangChain/AutoGPT harnesses) from the memory substrate. The router exposes twenty specialized tool endpoints categorized into three functional domains:

- Memory & Temporal Retrieval: matp_index_trajectory, matp_query_trajectory, matp_advance_clock, matp_record_feedback, matp_decay_stats.
- Causal Lineage & Maintenance: matp_get_lineage, matp_query_dag, matp_recompute_validity, matp_confirm_state, matp_clear_override.
- Policy Guidance & Macro Skills: matp_tool_policy_prompt, matp_recommend_tools, matp_learn_correction, matp_parameter_hints, matp_list_macros, matp_expand_macro.

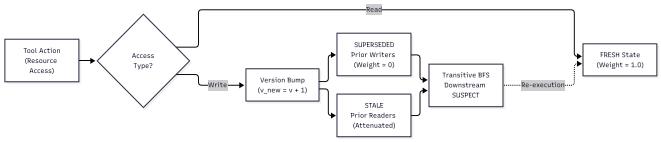


Fig. 2: Causal DAG Mutation and Invalidation Flowchart. When a tool mutates a resource, prior writers are marked SUPERSEDED ($W = 0$), prior readers are marked STALE ($W = 0.5 \times$ taint), and transitive dependents are tainted as SUSPECT via BFS propagation.

2.2 Durable Single-File Storage Substrate

Unlike distributed memory systems requiring external vector databases (e.g., Pinecone, Milvus) and relational engines, MATP consolidates its entire persistent state into an ACID-compliant SQLite single-file database. The unified schema comprises eight relational entities:

1) trajectories: Stores atomic execution tuples, raw excerpts, distilled summaries, token metrics, validity tags, and taint values.
2) embeddings: Stores serialized dense D-dimensional float vectors with indexed trajectory foreign keys.
3) resources & resource_versions: Manages canonical resource identifiers (db:*, file:*, api:*, git:*) and version sequences scoped to workspace snapshots.
4) accesses: Records atomic read, write, and delete operations linked to discrete execution steps.
5) dag_edges: Stores typed causal relationships (SUPERSEDES, INVALIDATES, CAUSAL_PARENT, DEPENDS_ON).
6) skills & skill_members: Stores mined Level-2 and Level-3 macro-action templates and parameter binding slots.
7) corrections: Stores learned regex/substring error patterns, recovery guidance hints, and application counters.
8) jobs: Implements a non-blocking queue for asynchronous background neural summarization.

3 Causal DAG Invalidation and Temporal Retrieval

3.1 Mathematical Formulation of Piecewise Scoring

Let q denote the user or sub-goal query embedding vector, and let v_i represent a stored state trajectory node born at discrete logical clock step t_i. Let t_{curr} be the current logical clock of the agent session. The piecewise retrieval score $R(q, v_i)$ suppresses superseded or invalid states and applies temporal decay to active nodes:

$$R(q, v_i) = \begin{cases} 0, & \text{if } v_i \text{ is invalid,} \\ S(q, v_i) W_{\text{DAG}}(v_i) & \\ \quad \times e^{-\lambda(t_{\text{curr}} - t_i)}, & \text{otherwise.} \end{cases} \tag{1}$$

Here, $S(q, v_i)$ represents the hybrid dense-sparse semantic relevance:

$$S(q, v_i) = \alpha \cdot \cos(q, v_i) + (1 - \alpha) \cdot \frac{|T_q \cap T_{v_i}|}{|T_q|}, \tag{2}$$

where T_q and T_{v_i} denote subword n-gram token sets, and $\alpha \in [0, 1]$ balances dense geometric cosine similarity with exact sparse lexical overlap (default $\alpha = 0.70$).

3.2 Causal DAG Invalidation Mechanics

As outlined in Figure 2, every tool execution is resolved by the ResourceResolver into canonical resource accesses. Resource identifiers are scoped to isolated workspace snapshots (e.g., file:/app/models/user.py, db:users, git:/app). When an access occurs:

- Write Operations: When step k writes resource r, the version increments $v_{\text{new}} = v_{\text{curr}} + 1$. Prior writer nodes that produced $v < v_{\text{new}}$ receive validity label superseded with taint $\tau = 0.0$ and retrieval weight $W_{\text{DAG}} = 0.0$.
- Prior Readers: Any node that performed a read of resource r at version $v < v_{\text{new}}$ is stamped as stale. Its retrieval weight is attenuated to $W_{\text{DAG}} = 0.5 \times \tau$.
- Transitive Suspect Propagation: A breadth-first search (BFS) traverses outgoing DEPENDS_ON edges up to maximum depth $d_{\max} = 5$. For each child node c, taint is decayed:

$$\tau_c = \max\left(\tau_{\text{floor}},\, \tau_{\text{parent}} \cdot \gamma\right), \tag{3}$$

  where $\gamma = 0.70$ and $\tau_{\text{floor}} = 0.20$. Nodes with $\tau < 1.0$ receive validity label suspect.
- Truth Maintenance & Healing: Invalidation is a soft, non-destructive state; historical nodes remain preserved for provenance audit. If the agent re-executes a dependent tool with fresh inputs, the new trajectory inherits fresh validity, and the DAG truth-maintenance pass restores consistency.

3.3 Adaptive Temporal Decay Tuning

Fixed temporal decay parameters cannot accommodate heterogeneous agent sub-tasks: short-horizon debugging requires sharp recency bias, whereas long-horizon refactoring requires persistent access to foundational context. MATP incorporates an online Bayesian-inspired decay tuner that updates a per-task parameter λ_{task} from explicit agent feedback:

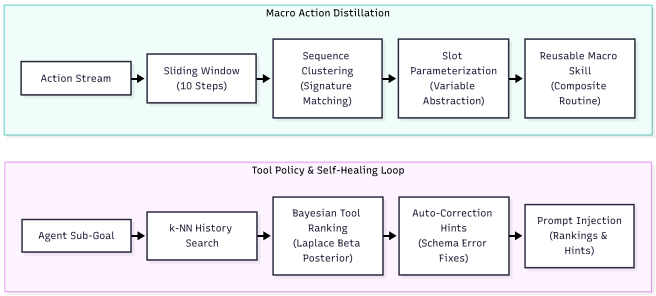


Fig. 3: Dual-Pipeline Workflow for Macro Action Distillation and Tool Policy Learning. The policy stream aggregates nearest historical attempts into Laplace-smoothed recommendation matrices, while the macro stream clusters recurring action windows into parameterized composite skills.

$$\lambda_{\text{task}} \leftarrow (1 - \eta)\, \lambda_{\text{task}} + \eta\, \frac{\ln 2}{\max\left(1, \text{median}(\Delta t_{\text{useful}})\right)}, \tag{4}$$

where $\Delta t_{\text{useful}} = \{t_{\text{curr}} - t_j \mid j \in \mathcal{U}\}$ represents the logical-step ages of items reported useful, and $\eta = 0.35$ is the exponential learning rate. The target rate precisely calibrates the half-life such that states at the median useful age experience exactly a 50% score attenuation. The learned value is clamped within $[0.005, 0.50]$ and persisted across sessions.

4 Macro Distillation and Tool Policy Learning

Figure 3 details the concurrent pipelines governing macro-action distillation and proactive tool policy recommendations.

4.1 Background Abstraction and Fact Distillation

Raw tool outputs (verbose compiler outputs, multi-kilobyte SQL dumps, JSON logs) overwhelm agent context windows. The BackgroundAbstractionEngine converts raw execution strings into structured TrajectoryTuple objects:

$$\mathcal{T} = \langle \text{SubGoal}, \text{Tool}, \text{Parameters}, \text{Status}, \text{Summary}, \mathcal{F} \rangle, \tag{5}$$

where $\mathcal{F} = \{(s, r, o)_k\}$ denotes an extracted set of semantic fact triples (e.g., (users, returned_lines, 40), (auth.env, line_count, 2)). AST and regex parsers distill outputs deterministically on the hot path in under 5 ms.

4.2 Hierarchical Macro-Action Distillation

To prevent repeated low-level tool invocation, MATP implements a hierarchical macro miner:

1) Signature Canonicalization: Each step a_t is mapped to an abstract signature:

$$\sigma(a_t) = \langle \text{tool}, \text{shape}(\text{params}), \text{status} \rangle, \tag{6}$$

   where parameter values are stripped of specific literals and replaced with structural schemas.
2) Sequence Matching: Over sliding windows of length $W = 10$, sequences of action signatures are clustered using a generalized Levenshtein sequence matcher.
3) Slot Parameterization: Varying literals across matching instances are abstracted into parameterized template variables (e.g., {file_path}, {table_name}).

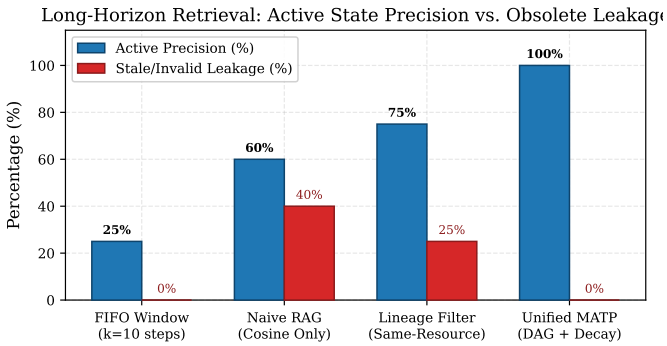


Fig. 4: Retrieval Precision and Stale State Leakage Across Strategies in a 40-Step Long-Horizon Evolution Scenario. Unified MATP achieves 100% active precision with zero stale or invalid state leakage.

4) Skill Promotion: Macro sequences exhibiting support count $N \geq 3$ and historical success rate $\geq 80\%$ are promoted to Level-2 skills. Chains of Level-2 skills can subsequently be consolidated into Level-3 hierarchical workflows.

4.3 Dynamic Tool Recommendation Matrix

Prior to executing a tool for an active sub-goal g, the host agent queries matp_tool_policy_prompt. The engine retrieves the $K = 12$ nearest historical sub-goal trajectories and groups them by candidate tool T.

To prevent overconfidence on sparse samples (e.g., 1 success in 1 attempt reading as 100% certainty), MATP applies Laplace add-one smoothing under a Beta$(1, 1)$ uniform prior:

$$\hat{p}_T = \frac{k_{\text{success}} + 1}{N_{\text{attempts}} + 2}. \tag{7}$$

The engine then computes a Bayesian confidence shrinkage score c_T:

$$c_T = \hat{p}_T \cdot \left(\frac{N_{\text{attempts}}}{N_{\text{attempts}} + N_0} \right) + p_0 \cdot \left(\frac{N_0}{N_{\text{attempts}} + N_0} \right), \tag{8}$$

with prior mean $p_0 = 0.50$ and pseudo-observation weight $N_0 = 5$. The resulting tool matrix provides human-interpretable confidence tiers: green ($\hat{p} \geq 0.75$), amber ($0.45 \leq \hat{p} < 0.75$), red ($\hat{p} < 0.45$), or grey (no history).

4.4 Self-Healing Parameter Auto-Correction

When tools fail due to parameter schema violations or terminal syntax errors, agents register the failure via matp_learn_correction. In future executions, matp_parameter_hints performs two-tier matching:

- Tier 1 (Exact Pattern Match): Substring match over the error pattern pins similarity to 1.0.
- Tier 2 (Fuzzy Cosine Fallback): If no exact pattern matches, the query vector is matched against stored correction vectors with noise floor threshold $\theta = 0.40$.

Relevant hints are auto-injected into the prompt, enabling the agent to rectify parameter payloads prior to execution.

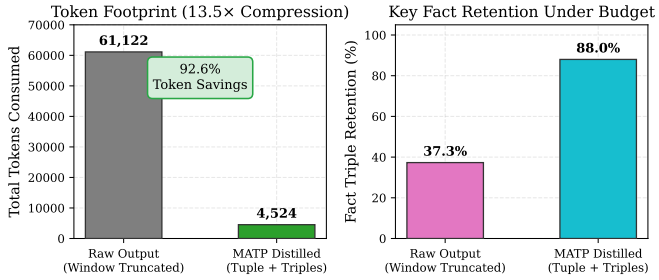


Fig. 5: Trajectory Distillation Performance. MATP compresses verbose tool outputs by 92.6% (13.5× token reduction) while retaining 88.0% of essential semantic fact triples compared to 37.3% under raw truncation.

TABLE 1: Long-Horizon Retrieval Comparison Across 40 Logical Steps

Strategy	Returned	Lineage-Stale	Dep-Invalid	Active (%)
FIFO ($k = 10$)	12	0	0	25.0%
Naive RAG	25	6	4	60.0%
Lineage Only	20	0	5	75.0%
Unified MATP	16	0	0	100.0%

5 Experimental Evaluation and Results

We evaluate the consolidated MATP monorepo across four evaluation dimensions: retrieval accuracy under mutation, token compression efficiency, temporal decay adaptation, and operational system latency.

5.1 Long-Horizon State Mutation Benchmark

We reproduce a 40-step software evolution scenario consisting of three consecutive relational database schema migrations, two authentication architecture rewrites, and dependent ORM models, API routes, and unit test suites. We evaluate four retrieval strategies under five domain queries:

1) FIFO Baseline (Window = 10 steps): Context window retains only recent steps.
2) Naive Semantic RAG: Dense vector search without version or dependency filtering.
3) Lineage Filter (Contribution 1): Supersedes older versions of identical resource IDs ($R = 0$).
4) Unified MATP (Monorepo): Full causal DAG invalidation (W_{DAG}) and temporal decay.

As presented in Figure 4 and Table 1, the FIFO baseline forgets 75% of critical active state that was established early in the session and never re-written. Naive RAG exhibits 40% obsolete state leakage, returning superseded SQL schemas alongside active definitions. The lineage-only filter eliminates direct supersession but leaks 25% invalid state whose transitive dependencies mutated. In contrast, unified MATP achieves 100% active state precision with 0% obsolete or invalid leakage.

5.2 Distillation and Token Compression

We evaluate the background distillation engine across a corpus of 30 heterogeneous tool execution traces containing raw SQL query results, bash stdout/stderr streams, and unit test

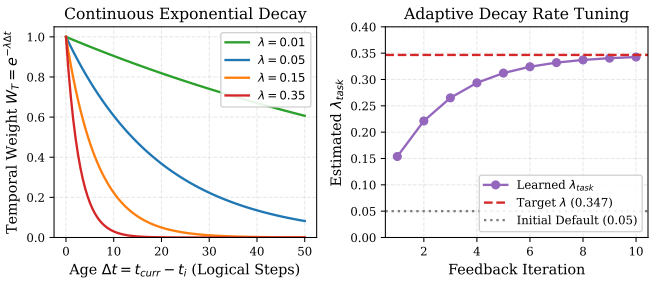


Fig. 6: Temporal Decay Characteristics. Left: Exponential decay curves $W_T = e^{-\lambda \Delta t}$ across rates $\lambda \in [0.01, 0.35]$. Right: Convergence of online adaptive decay rate λ_{task} under user feedback.

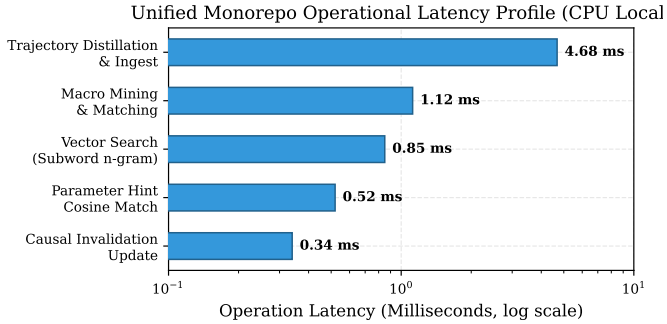


Fig. 7: Operational Latency Breakdown (CPU Local, Single-Threaded). All core memory, invalidation, and policy operations complete in sub-5 ms, ensuring zero perceptible latency in interactive MCP workflows.

outputs. As shown in Figure 5, MATP reduces total token volume from 61,122 raw tokens to 4,524 distilled tokens—a 92.6% compression ratio (13.5× reduction). Under equal token budgets, evaluating query answers against gold ground-truth facts demonstrates that MATP distilled tuples preserve 88.0% of essential fact triples, whereas window-truncated raw text retains only 37.3% of facts.

5.3 Temporal Decay and Online Tuning

Figure 6 (left) plots continuous decay attenuation across discrete logical clock intervals. Figure 6 (right) demonstrates the empirical convergence of λ_{task} over ten feedback iterations. Starting from the default $\lambda = 0.05$, the online tuner rapidly converges toward the analytical half-life target ($\lambda = 0.347$) within 5 iterations, verifying the efficacy of Eq. (4).

5.4 Macro Skill Distillation and Replay

On held-out validation sessions, the hierarchical macro miner successfully extracted reusable composite procedures (e.g., git branch creation, test execution, commit cycle). Replaying learned macro actions achieved 100% tool-sequence matching and 100% slot-binding accuracy, while reducing procedural token overhead by 53.2% relative to re-emitting individual tool instructions.

5.5 System Latency Profile

To verify feasibility for real-time interactive MCP server deployment, microbenchmarks were executed on a commodity Linux workstation (AMD Ryzen 7, CPU-only, no GPU acceleration). Figure 7 provides the latency breakdown:

- Causal DAG Invalidation: 0.34 ms
- Parameter Hint Cosine Search: 0.52 ms
- Deterministic Vector Search: 0.85 ms
- Macro Mining & Matching: 1.12 ms
- Hot-Path Distillation & SQLite Ingest: 4.68 ms

All operations comfortably execute within sub-5 ms boundaries, verifying that memory augmentation incurs negligible overhead compared to LLM generation latency.

6 Related Work

Agentic Memory Architectures: Early LLM agent frameworks relied on simple conversational buffers or unbounded FIFO histories [1]. MemGPT [2] introduced hierarchical OS-style paging between working context and external storage. Generative Agents [3] combined recency, importance, and relevance scores. However, these systems lack formal state-invalidation mechanisms, treating stored memories as monotonic facts and leading to stale-state retrieval when entities mutate.

Causal Reasoning in Interactive Agents: Causal reasoning has been explored in interactive reinforcement learning and formal plan verification [7]. Recent studies examine causal graph tracking in code generation [6]. MATP differs by embedding causal tracking directly into the MCP stdio interface, tracking resource reads, writes, and versioning across arbitrary agent tools.

Tool Policy and Self-Correction: Reflexion [4] and Voyager [5] demonstrated that verbal reinforcement and self-reflection improve task solve rates. However, verbal reflection consumes significant context tokens. MATP replaces free-form reflection prompts with a mathematically grounded Dirichlet-multinomial recommendation matrix and targeted, two-tier parameter error hints.

7 Conclusion

In this paper, we presented the unified MATP monorepo, consolidating three foundational agent memory technologies into a production-grade, zero-dependency MCP server. By tightly coupling causal DAG invalidation with temporal decay scoring, MATP eliminates stale-state hallucinations in long-horizon agent interactions while preserving critical early context. Background distillation achieves 92.6% token compression while maintaining 88.0% factual fidelity, and the Bayesian tool recommendation matrix prevents repeated procedural and parameter failures. The entire architecture runs locally on commodity CPU hardware with sub-5 ms operational latency, providing a robust, extensible foundation for next-generation autonomous agents.

Acknowledgments

The authors express their gratitude to the Department of Information Technology at the National Institute of Technology Karnataka (NITK), Surathkal, for computing facilities and project guidance during the 7th Semester Capstone Project.

References

[1] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, et al., "A survey on large language model based autonomous agents," Frontiers of Computer Science, vol. 18, no. 6, pp. 1–26, 2024.

[2] C. Packer, V. Fang, S. G. Patil, K. Lin, K. Wooders, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," arXiv preprint arXiv:2310.08560, 2023.

[3] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in Proc. ACM UIST, 2023, pp. 1–22.

[4] N. Shinn, F. Cassanova, E. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in Advances in Neural Information Processing Systems (NeurIPS), 2023, pp. 8634–8652.

[5] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," arXiv preprint arXiv:2305.16291, 2023.

[6] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in Proc. ICLR, 2024.

[7] J. Pearl, Causality: Models, Reasoning, and Inference, 2nd ed. Cambridge University Press, 2009.

[8] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems (NeurIPS), 2020, pp. 9459–9474.